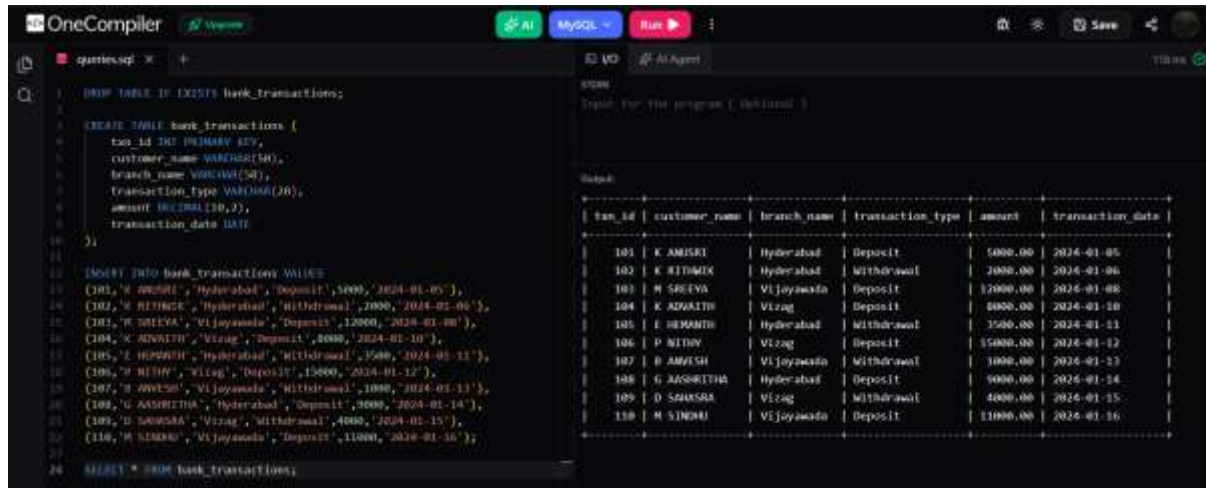


AGGREGATE FUNCTIONS with WHERE, GROUP BY, HAVING, ORDER BY

STEP 1 — Create Bank Transactions Table

Create and Populate Bank Transactions Table

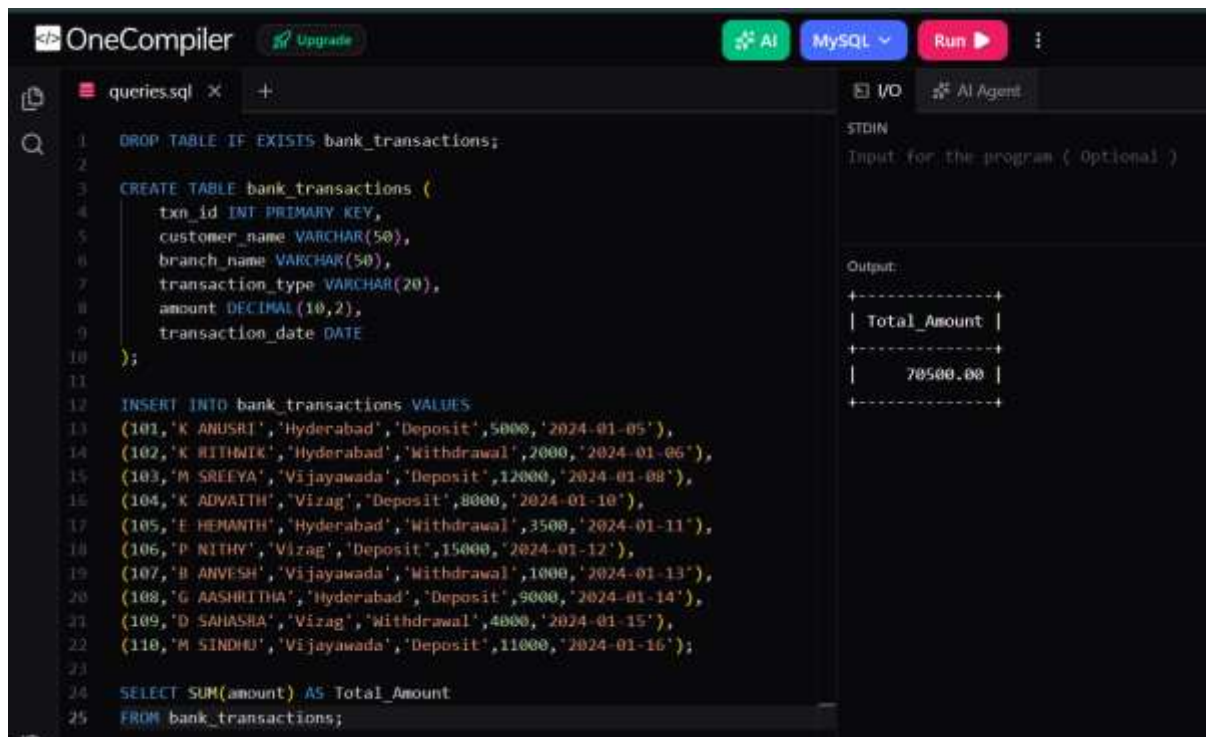


```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4   txn_id INT PRIMARY KEY,
5   customer_name VARCHAR(50),
6   branch_name VARCHAR(50),
7   transaction_type VARCHAR(20),
8   amount DECIMAL(10,2),
9   transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT * FROM bank_transactions;
```

txn_id	customer_name	branch_name	transaction_type	amount	transaction_date
101	K ANUSRI	Hyderabad	Deposit	5000.00	2024-01-05
102	K RITHVIK	Hyderabad	Withdrawal	2000.00	2024-01-06
103	M SREEYA	Vijayawada	Deposit	12000.00	2024-01-08
104	K ADVAITH	Vizag	Deposit	8000.00	2024-01-10
105	E HEMANTH	Hyderabad	Withdrawal	3500.00	2024-01-11
106	P NITHY	Vizag	Deposit	15000.00	2024-01-12
107	B ANVESH	Vijayawada	Withdrawal	1000.00	2024-01-13
108	G AASHRITHA	Hyderabad	Deposit	9000.00	2024-01-14
109	D SAHASRA	Vizag	Withdrawal	4000.00	2024-01-15
110	M SINDHU	Vijayawada	Deposit	11000.00	2024-01-16

Q1 — SUM()

SUM() — Find the Total Amount Transacted

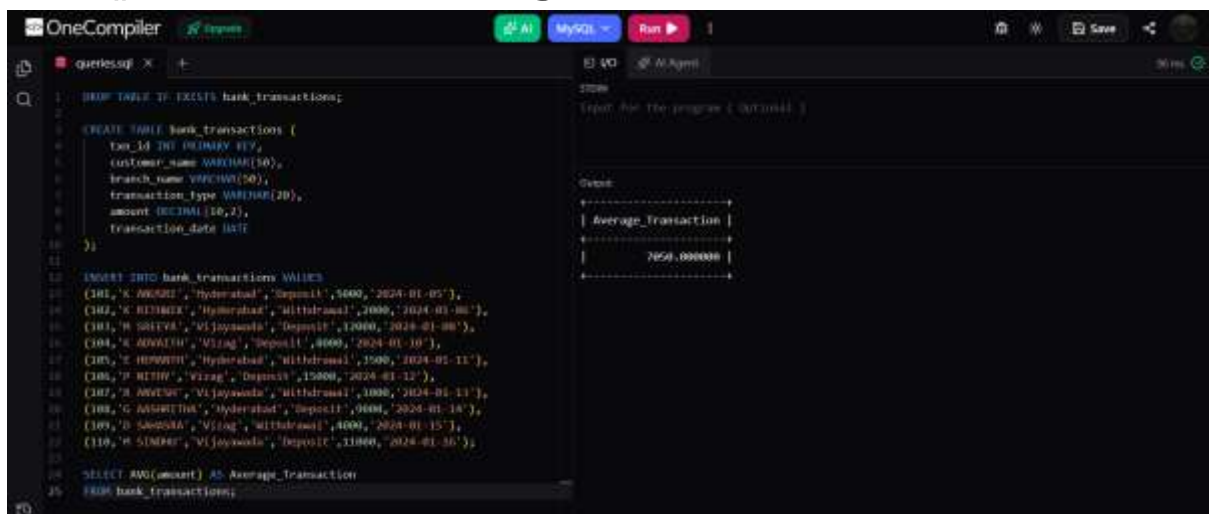


```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4   txn_id INT PRIMARY KEY,
5   customer_name VARCHAR(50),
6   branch_name VARCHAR(50),
7   transaction_type VARCHAR(20),
8   amount DECIMAL(10,2),
9   transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT SUM(amount) AS Total_Amount
25 FROM bank_transactions;
```

Total_Amount
70500.00

Q2 — AVG()

AVG() — Find the Average Transaction Amount



The screenshot shows the OneCompiler interface with a MySQL database. The code in the editor defines a table `bank_transactions` with columns `txn_id`, `customer_name`, `branch_name`, `transaction_type`, `amount`, and `transaction_date`. It inserts 10 rows of transaction data and then queries for the average transaction amount.

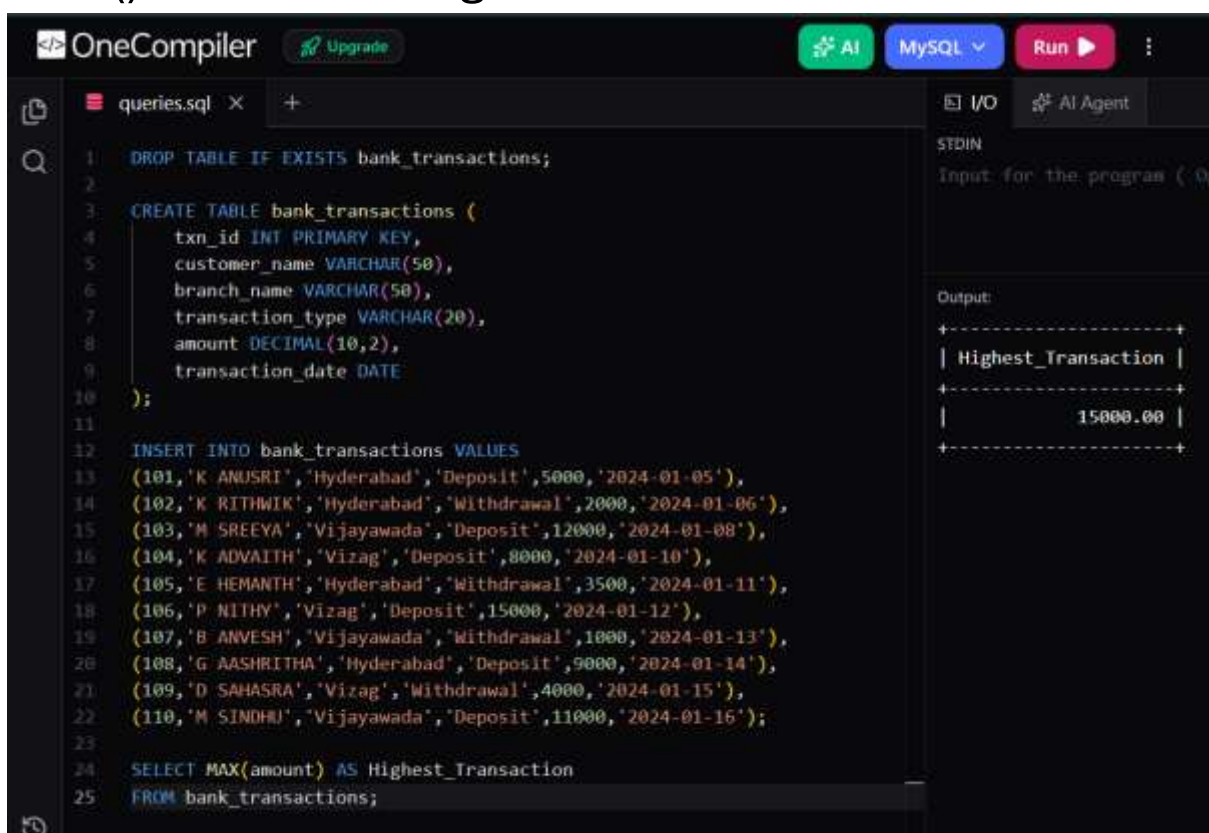
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT AVG(amount) AS Average_Transaction
25 FROM bank_transactions;
```

The output on the right shows the result of the query:

Average_Transaction
7650.000000

Q3 — MAX()

MAX() — Find the Highest Transaction Amount



The screenshot shows the OneCompiler interface with a MySQL database. The code in the editor defines a table `bank_transactions` with columns `txn_id`, `customer_name`, `branch_name`, `transaction_type`, `amount`, and `transaction_date`. It inserts 10 rows of transaction data and then queries for the highest transaction amount.

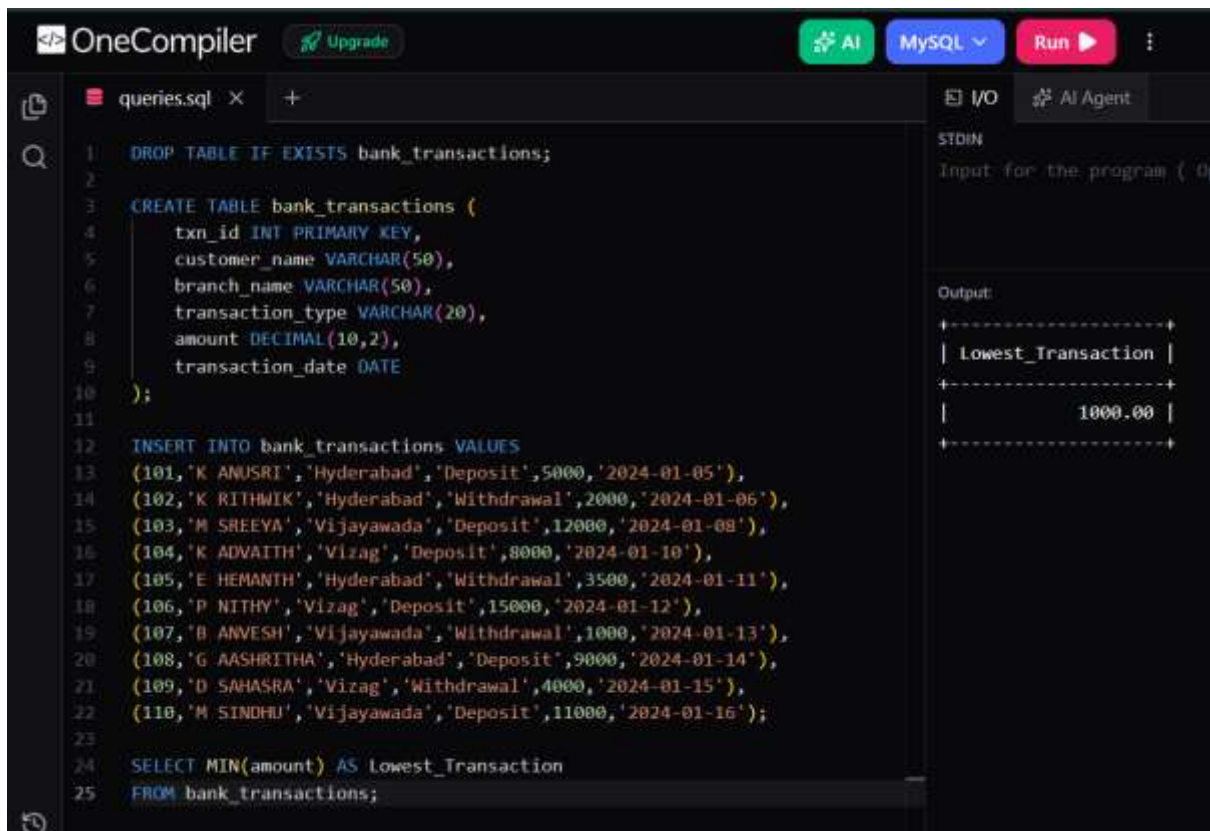
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT MAX(amount) AS Highest_Transaction
25 FROM bank_transactions;
```

The output on the right shows the result of the query:

Highest_Transaction
15000.00

Q4 — MIN()

MIN() — Find the Lowest Transaction Amount



The screenshot shows the OneCompiler MySQL IDE. The main editor displays SQL code for creating a table, inserting data, and finding the minimum transaction amount. The right sidebar shows the output of the query, which is 1000.00.

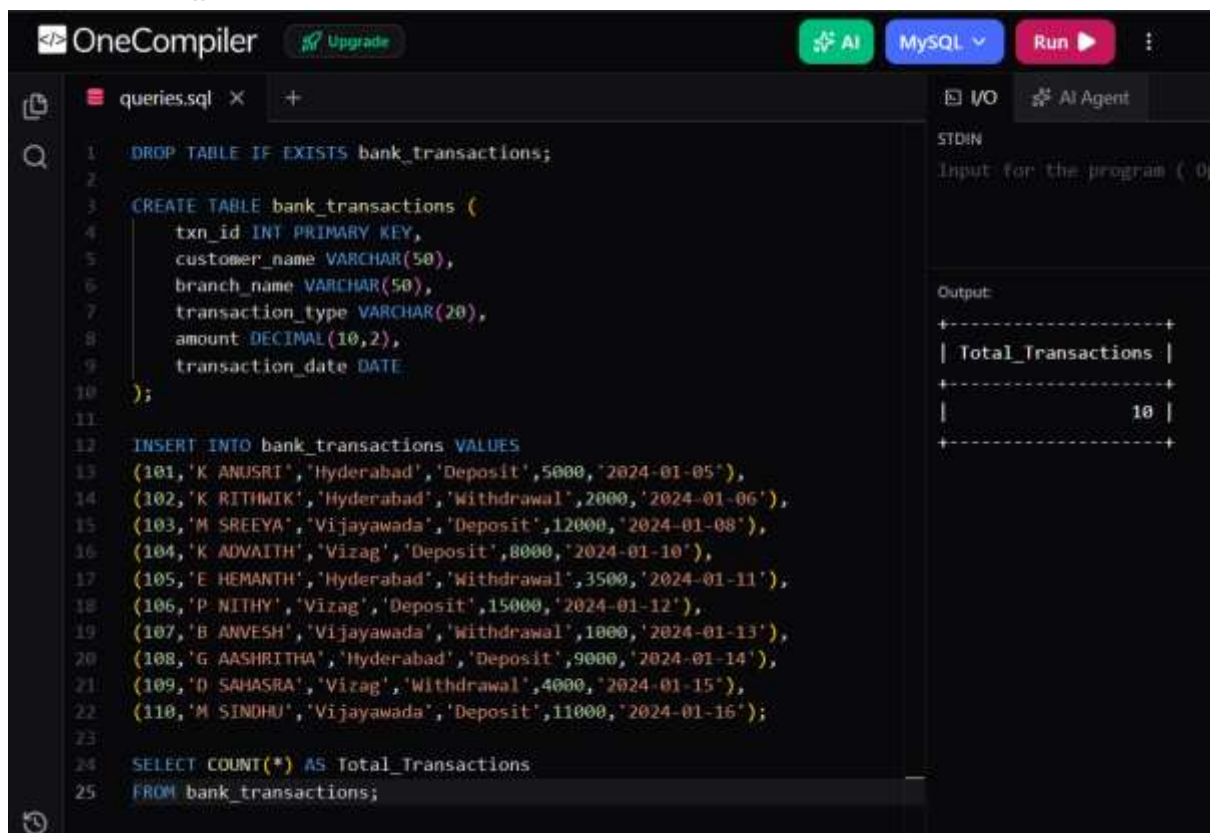
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHWIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT MIN(amount) AS Lowest_Transaction
25 FROM bank_transactions;
```

Output:

Lowest_Transaction
1000.00

Q5 — COUNT()

COUNT() — Find the Total Number of Transactions



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in a file named 'queries.sql'. The script defines a 'bank_transactions' table with columns: 'txn_id' (INT PRIMARY KEY), 'customer_name' (VARCHAR(50)), 'branch_name' (VARCHAR(50)), 'transaction_type' (VARCHAR(20)), 'amount' (DECIMAL(10,2)), and 'transaction_date' (DATE). It then inserts 10 rows of transaction data. Finally, it executes a query to count the total number of transactions, aliasing the result as 'Total_Transactions'.

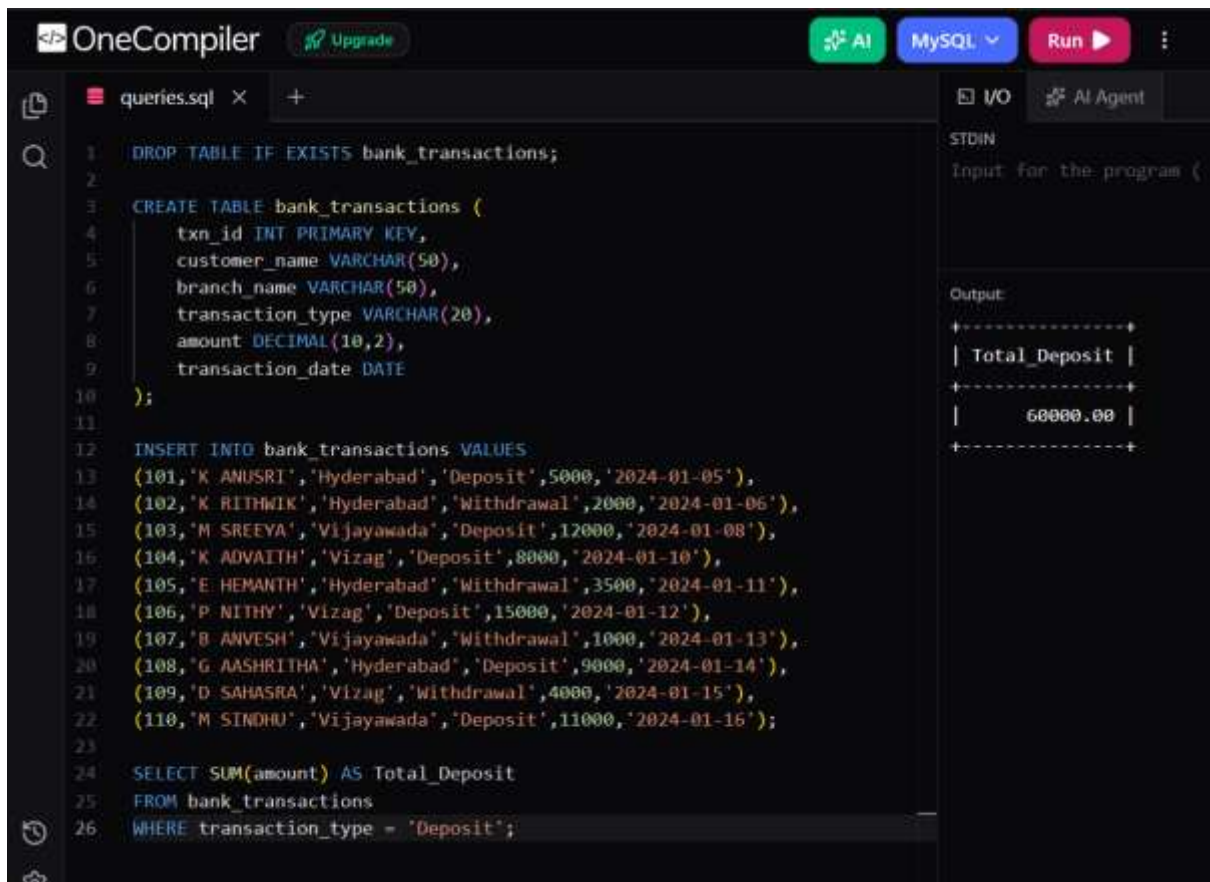
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101, 'K ANUSRI', 'Hyderabad', 'Deposit', 5000, '2024-01-05'),
14 (102, 'K RITHWIK', 'Hyderabad', 'Withdrawal', 2000, '2024-01-06'),
15 (103, 'M SREEYA', 'Vijayawada', 'Deposit', 12000, '2024-01-08'),
16 (104, 'K ADVAITH', 'Vizag', 'Deposit', 8000, '2024-01-10'),
17 (105, 'E HEMANTH', 'Hyderabad', 'Withdrawal', 3500, '2024-01-11'),
18 (106, 'P NITHY', 'Vizag', 'Deposit', 15000, '2024-01-12'),
19 (107, 'B ANVESH', 'Vijayawada', 'Withdrawal', 1000, '2024-01-13'),
20 (108, 'G AASHRITHA', 'Hyderabad', 'Deposit', 9000, '2024-01-14'),
21 (109, 'D SAHASRA', 'Vizag', 'Withdrawal', 4000, '2024-01-15'),
22 (110, 'M SINDHU', 'Vijayawada', 'Deposit', 11000, '2024-01-16');
23
24 SELECT COUNT(*) AS Total_Transactions
25 FROM bank_transactions;
```

The right pane shows the output of the query. Under the 'Output' section, a table displays the result:

Total_Transactions
10

Q6 — WHERE + SUM()

WHERE + SUM() — Find the Total Amount Deposited



The screenshot shows the OneCompiler MySQL IDE interface. The main editor displays a SQL script with the following content:

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT SUM(amount) AS Total_Deposit
25 FROM bank_transactions
26 WHERE transaction_type = 'Deposit';
```

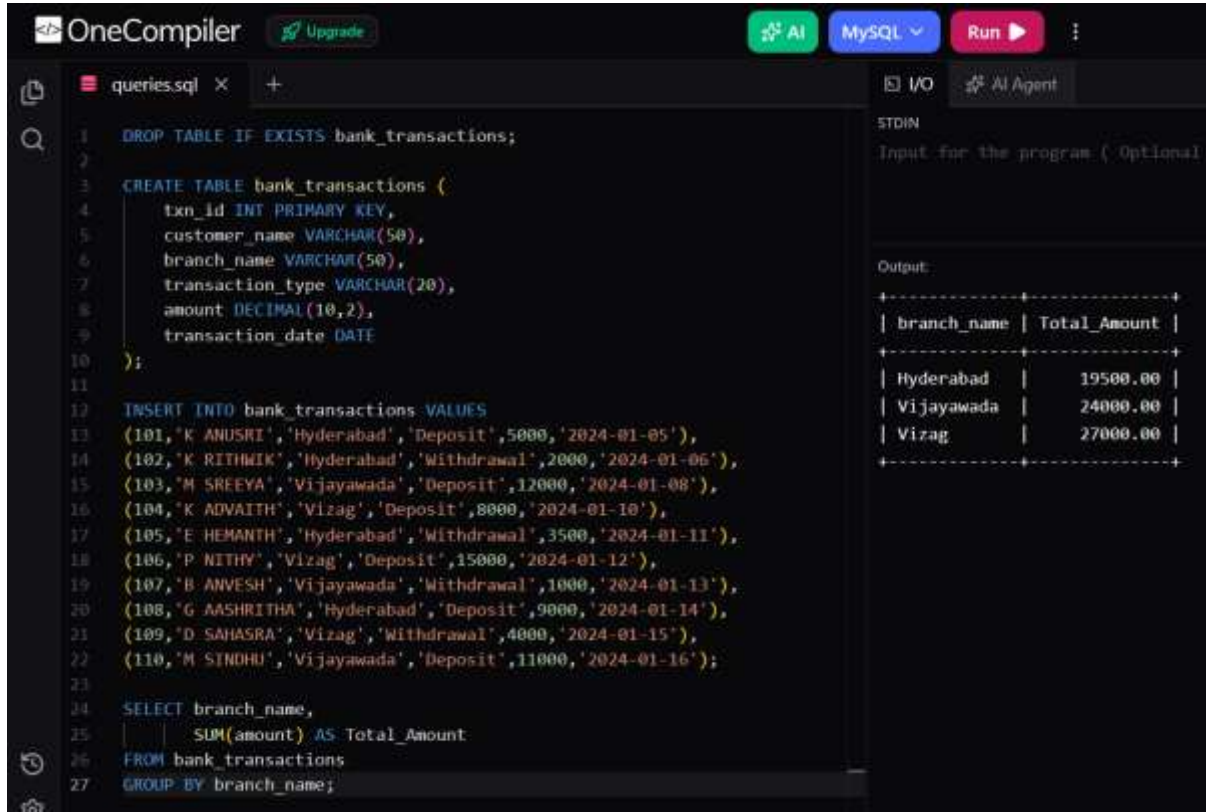
On the right side, the I/O panel shows the output of the query:

```
STDIN
Input for the program (

Output:
+-----+
| Total_Deposit |
+-----+
|      60000.00 |
+-----+
```

Q7 — GROUP BY

GROUP BY — Find Total Transaction Amount for Each Branch



The screenshot shows the OneCompiler MySQL IDE. The left pane contains the following SQL code:

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHMIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT branch_name,
25        SUM(amount) AS Total_Amount
26 FROM bank_transactions
27 GROUP BY branch_name;
```

The right pane shows the output of the query:

STDIN
Input for the program (Optional)

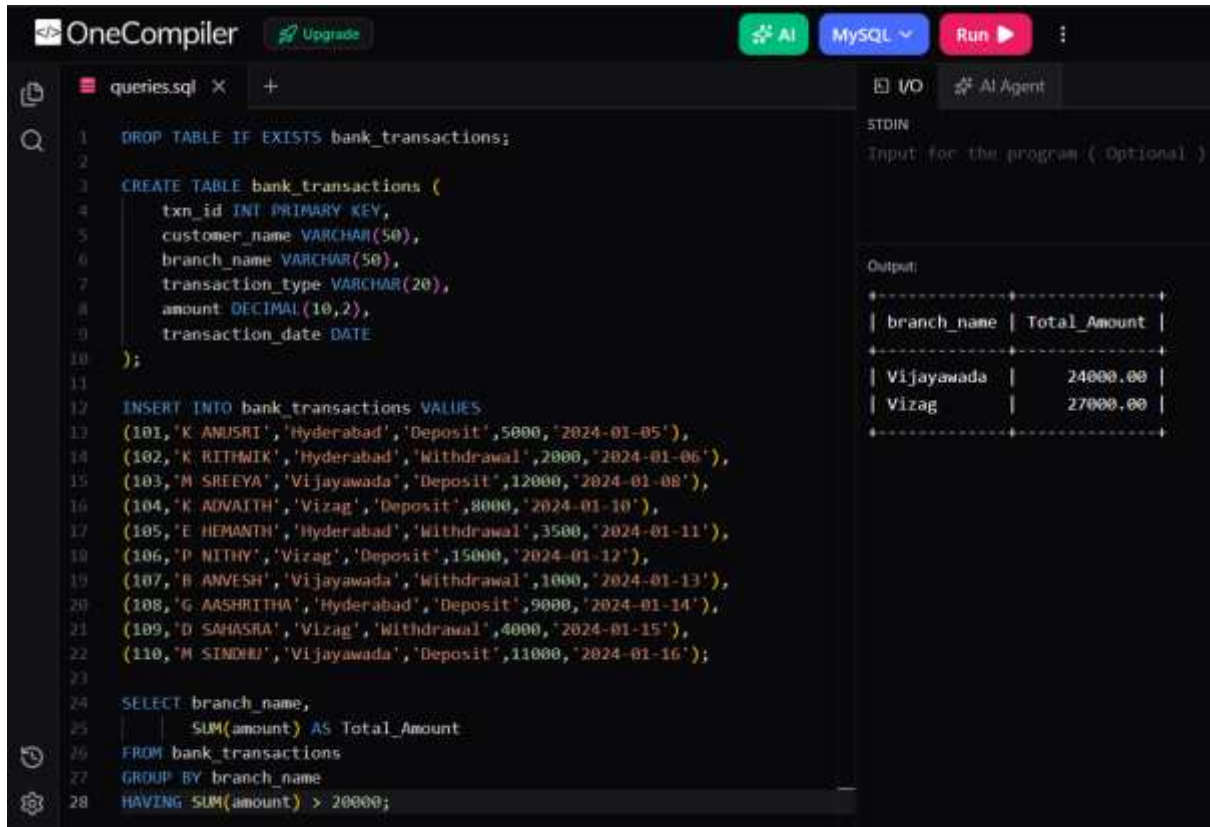
Output:

branch_name	Total_Amount
Hyderabad	19500.00
Vijayawada	24000.00
Vizag	27000.00

Q8 — GROUP BY + HAVING

GROUP BY + HAVING — Find Branches Whose Total

Transaction Amount Exceeds \$20,000



The screenshot shows the OneCompiler MySQL IDE interface. The main editor displays a SQL script with the following queries:

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANJUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT branch_name,
25        SUM(amount) AS Total_Amount
26 FROM bank_transactions
27 GROUP BY branch_name
28 HAVING SUM(amount) > 20000;
```

The right sidebar shows the output of the query, displaying a table with the following data:

branch_name	Total_Amount
Vijayawada	24000.00
Vizag	27000.00

Q9 — GROUP BY + ORDER BY

GROUP BY + ORDER BY — Rank Branches from

Highest to Lowest Transaction Volume

OneCompiler

Upgrade

AI

MySQL

Run

queries.sql

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SANASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT branch_name,
25        SUM(amount) AS Total_Amount
26 FROM bank_transactions
27 GROUP BY branch_name
28 ORDER BY Total_Amount DESC;
```

STDIN

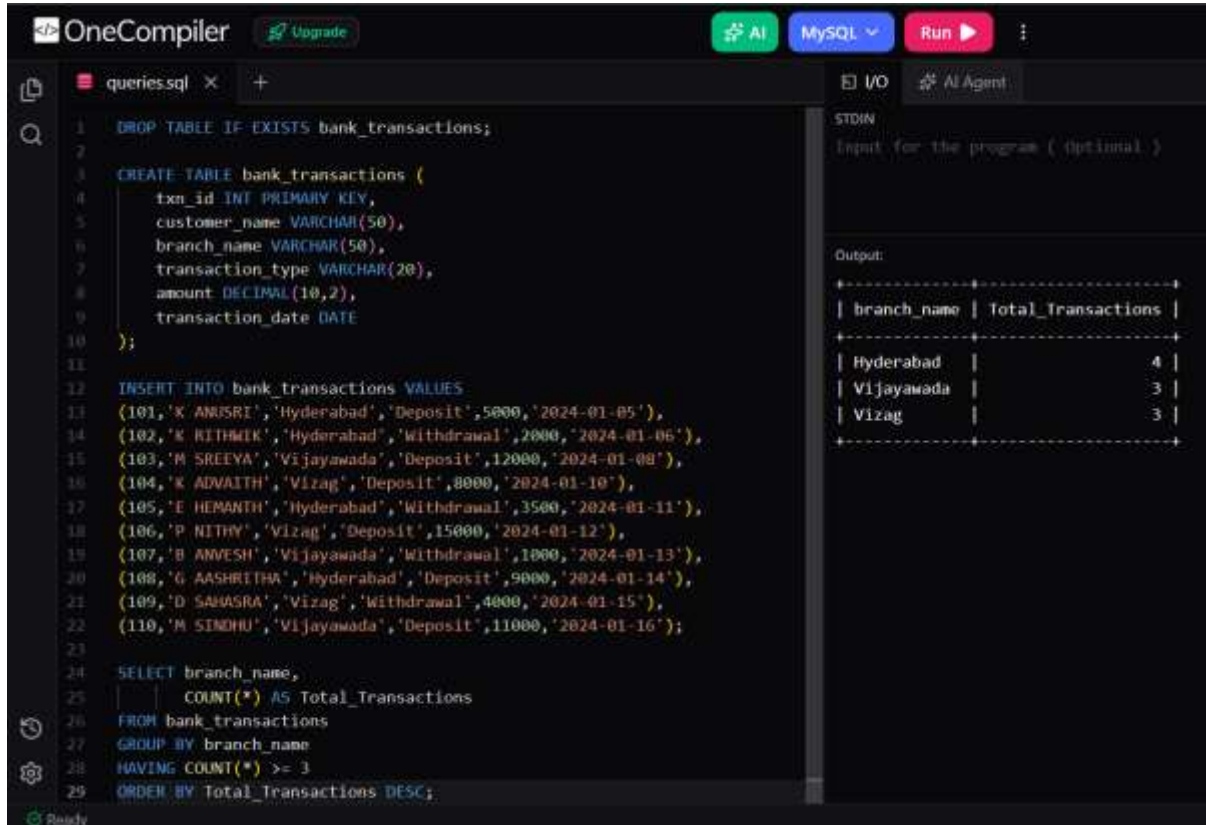
Input for the program (Optional)

Output:

branch_name	Total_Amount
Vizag	27000.00
Vijayawada	24000.00
Hyderabad	19500.00

Q10 — GROUP BY + HAVING + ORDER BY

GROUP BY + HAVING + ORDER BY — Rank Branches That Processed at Least 3 Transactions



The screenshot shows the OneCompiler IDE with a MySQL query in the editor. The query creates a table named `bank_transactions` with columns `txn_id` (INT PRIMARY KEY), `customer_name` (VARCHAR(50)), `branch_name` (VARCHAR(50)), `transaction_type` (VARCHAR(20)), `amount` (DECIMAL(10,2)), and `transaction_date` (DATE). It then inserts 10 transactions into the table. Finally, it runs a query to select the branch name and the count of transactions, grouped by branch name, with a HAVING clause to filter for branches with at least 3 transactions, and an ORDER BY clause to sort the results in descending order of total transactions.

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101, 'K ANRISRI', 'Hyderabad', 'Deposit', 5000, '2024-01-05'),
14 (102, 'K RITHMEK', 'Hyderabad', 'Withdrawal', 2000, '2024-01-06'),
15 (103, 'M SREEYA', 'Vijayawada', 'Deposit', 12000, '2024-01-08'),
16 (104, 'K ADVAITH', 'Vizag', 'Deposit', 8000, '2024-01-10'),
17 (105, 'E HEMANTH', 'Hyderabad', 'Withdrawal', 3500, '2024-01-11'),
18 (106, 'P NITHY', 'Vizag', 'Deposit', 15000, '2024-01-12'),
19 (107, 'B ANVESH', 'Vijayawada', 'Withdrawal', 1000, '2024-01-13'),
20 (108, 'G AASHRITHA', 'Hyderabad', 'Deposit', 9000, '2024-01-14'),
21 (109, 'D SAHASRA', 'Vizag', 'Withdrawal', 4000, '2024-01-15'),
22 (110, 'M SINDHU', 'Vijayawada', 'Deposit', 11000, '2024-01-16');
23
24 SELECT branch_name,
25        COUNT(*) AS Total_Transactions
26 FROM bank_transactions
27 GROUP BY branch_name
28 HAVING COUNT(*) >= 3
29 ORDER BY Total_Transactions DESC;
```

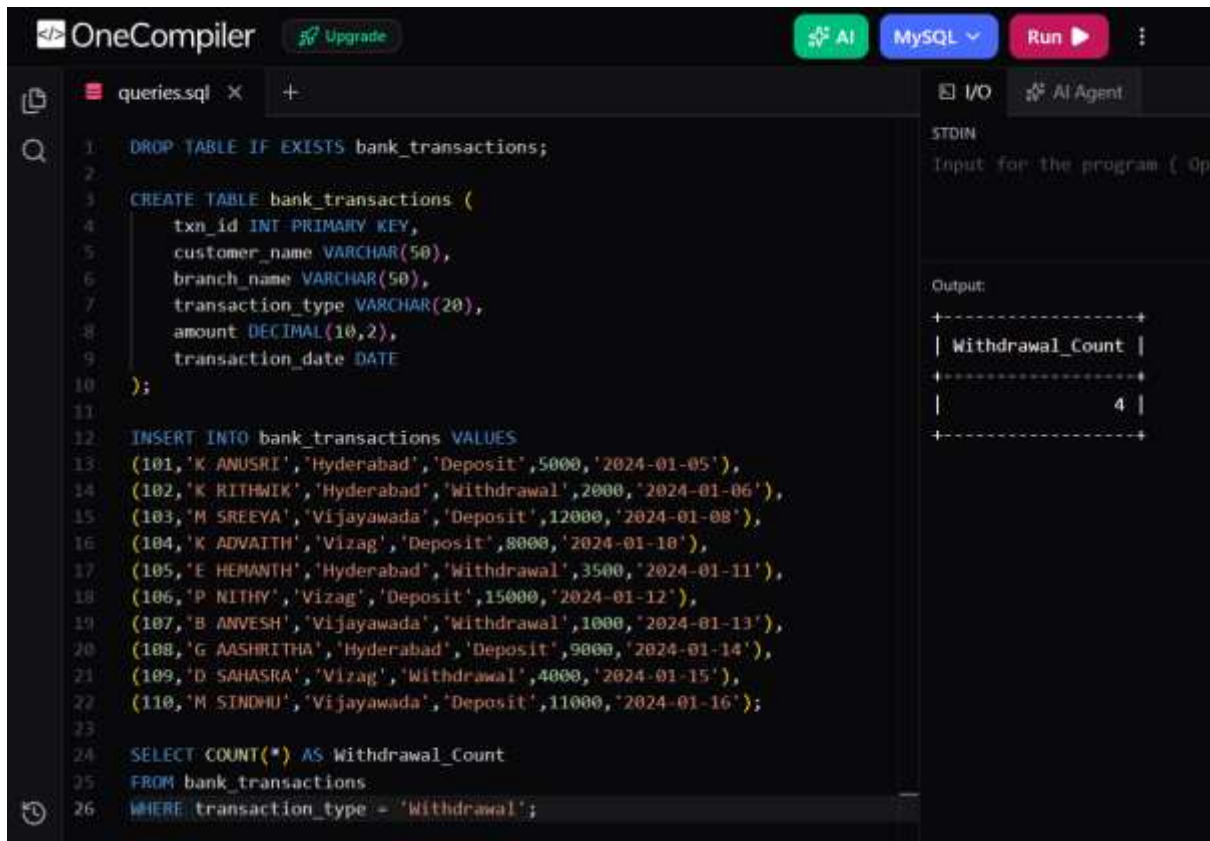
The output of the query is displayed on the right side of the IDE, showing a table with two columns: `branch_name` and `Total_Transactions`. The results are as follows:

branch_name	Total_Transactions
Hyderabad	4
Vijayawada	3
Vizag	3

Q11 — WHERE + COUNT()

WHERE + COUNT() — Count Withdrawal

Transactions



The screenshot shows the OneCompiler MySQL interface. The main editor displays a series of SQL queries to create a table, insert data, and query for withdrawal counts. The output panel on the right shows the result of the final query as a table with one row and one column.

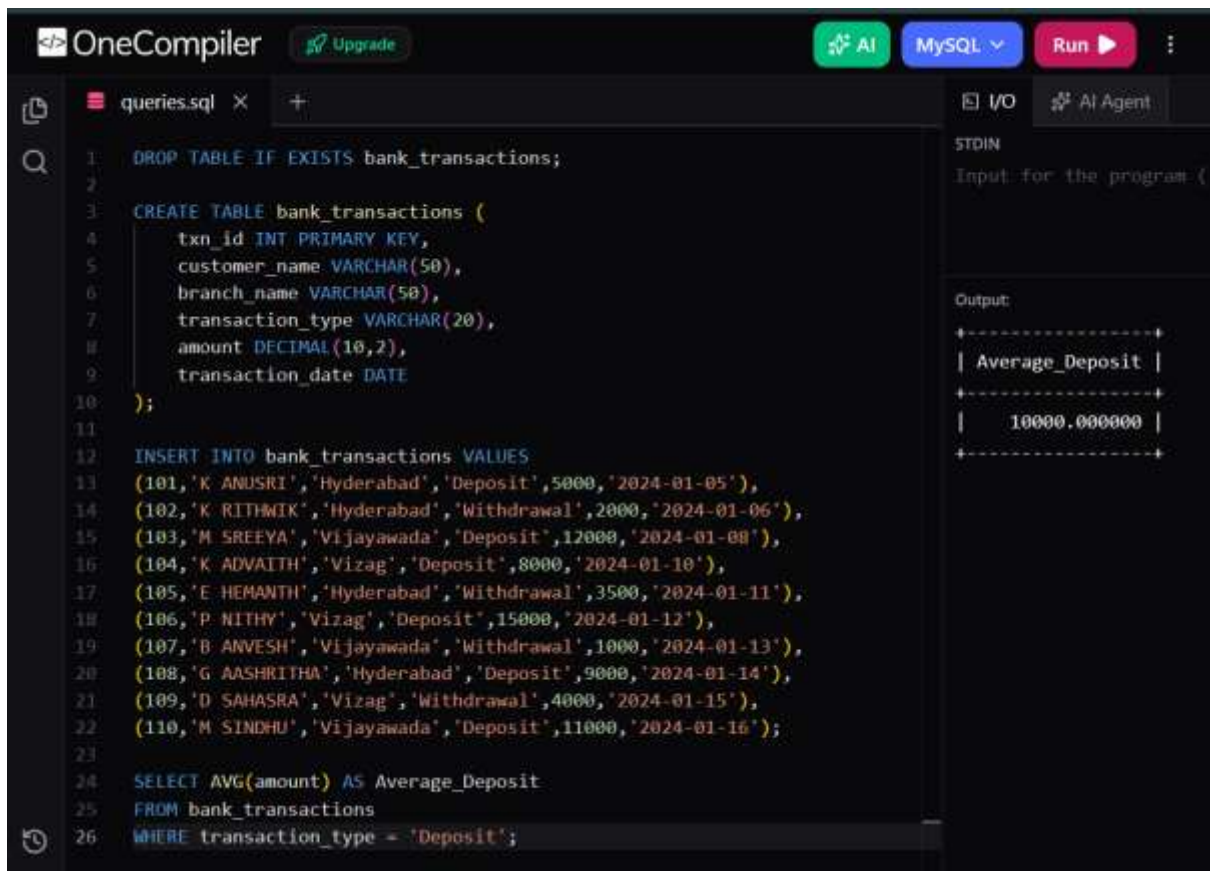
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K. ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K. RITHWIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M. SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K. ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E. HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P. NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B. ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G. AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D. SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M. SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT COUNT(*) AS Withdrawal_Count
25 FROM bank_transactions
26 WHERE transaction_type = 'Withdrawal';
```

Output:

Withdrawal_Count
4

Q12 — WHERE + AVG()

WHERE + AVG() — Find the Average Deposit Amount



The screenshot shows the OneCompiler MySQL IDE interface. The main editor displays a SQL script with the following queries:

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101, 'K ANUSRI', 'Hyderabad', 'Deposit', 5000, '2024-01-05'),
14 (102, 'K RITHVIK', 'Hyderabad', 'Withdrawal', 2000, '2024-01-06'),
15 (103, 'M SREEYA', 'Vijayawada', 'Deposit', 12000, '2024-01-08'),
16 (104, 'K ADVAITH', 'Vizag', 'Deposit', 8000, '2024-01-10'),
17 (105, 'E HEMANTH', 'Hyderabad', 'Withdrawal', 3500, '2024-01-11'),
18 (106, 'P NITHY', 'Vizag', 'Deposit', 15000, '2024-01-12'),
19 (107, 'B ANVESH', 'Vijayawada', 'Withdrawal', 1000, '2024-01-13'),
20 (108, 'G AASHRITHA', 'Hyderabad', 'Deposit', 9000, '2024-01-14'),
21 (109, 'D SAHASRA', 'Vizag', 'Withdrawal', 4000, '2024-01-15'),
22 (110, 'M SINDHU', 'Vijayawada', 'Deposit', 11000, '2024-01-16');
23
24 SELECT AVG(amount) AS Average_Deposit
25 FROM bank_transactions
26 WHERE transaction_type = 'Deposit';
```

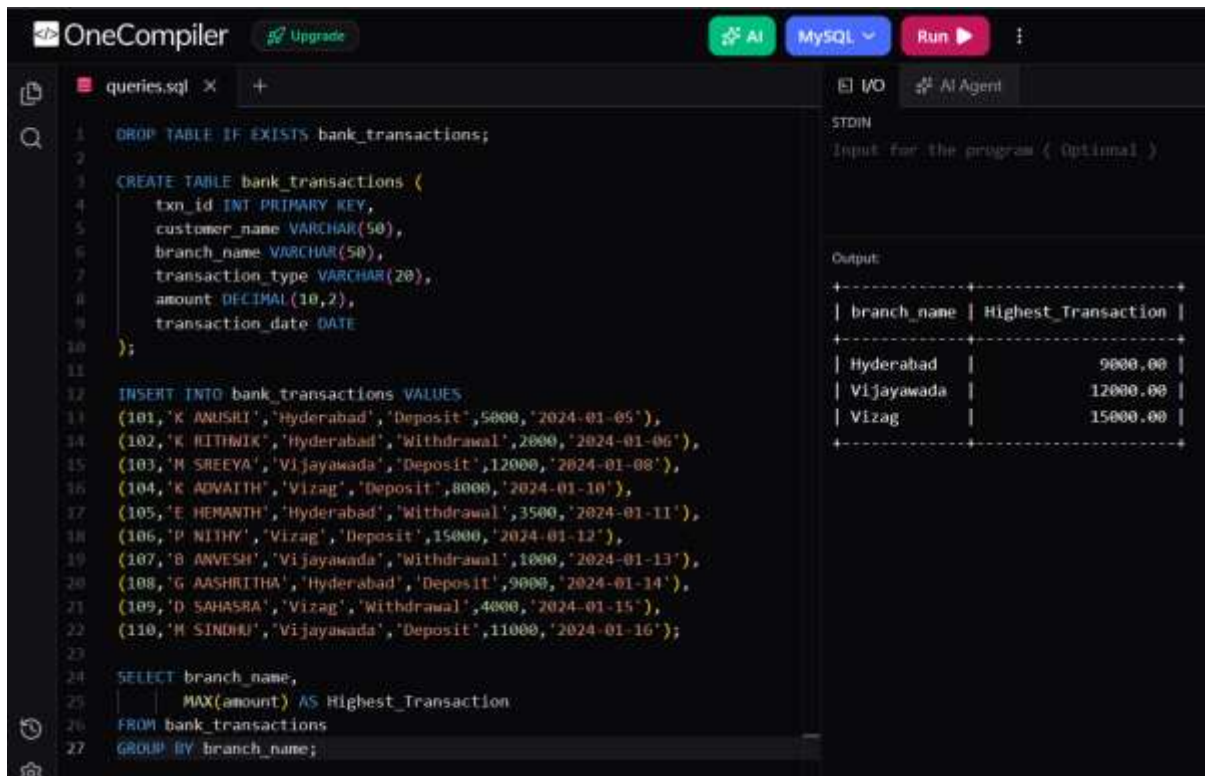
The right sidebar shows the output of the query:

```
Output:
+-----+
| Average_Deposit |
+-----+
| 10000.000000    |
+-----+
```

Q13 — GROUP BY + MAX()

GROUP BY + MAX() — Find the Highest Transaction

in Each Branch



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL query file named 'queries.sql' with the following code:

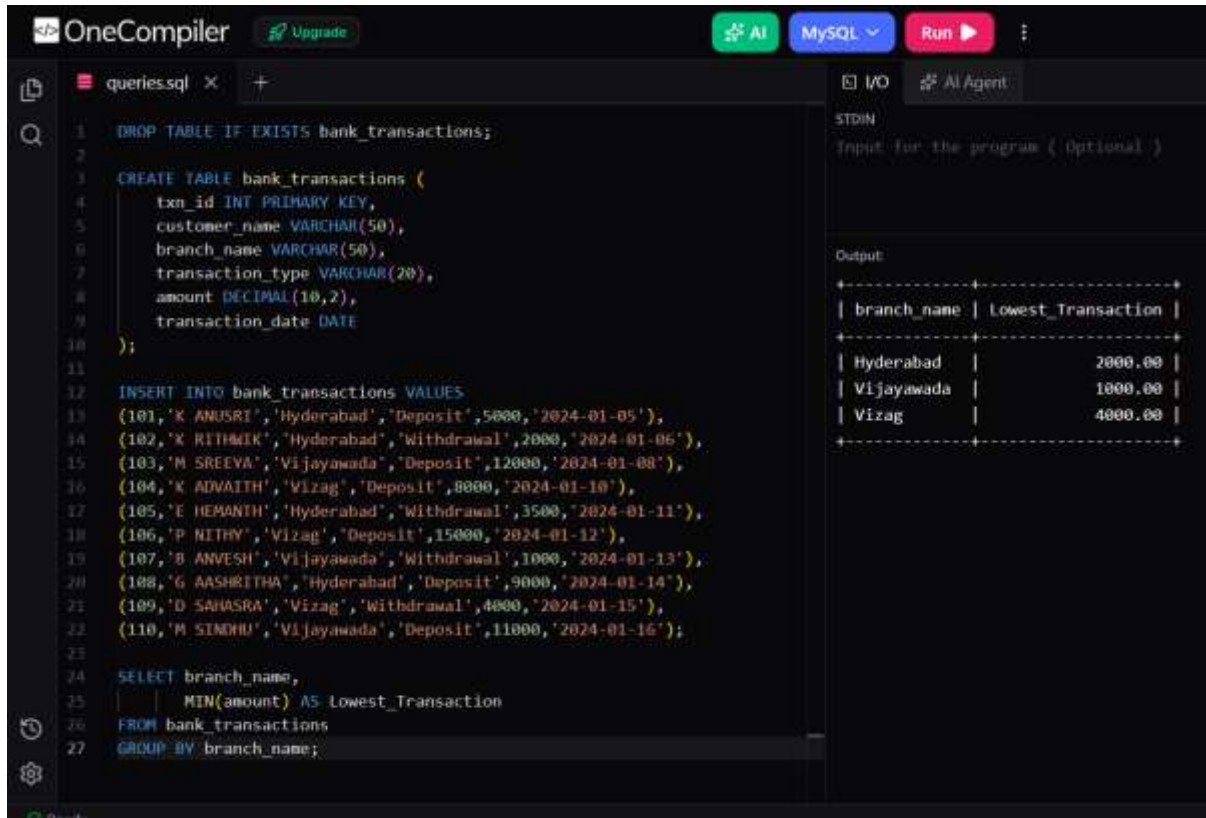
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101, 'K ANUSILI', 'Hyderabad', 'Deposit', 5000, '2024-01-05'),
14 (102, 'K RITHWIK', 'Hyderabad', 'Withdrawal', 2000, '2024-01-06'),
15 (103, 'M SREEYA', 'Vijayawada', 'Deposit', 12000, '2024-01-08'),
16 (104, 'K ADVAITH', 'Vizag', 'Deposit', 8000, '2024-01-10'),
17 (105, 'E HEMANTH', 'Hyderabad', 'Withdrawal', 3500, '2024-01-11'),
18 (106, 'P NITHY', 'Vizag', 'Deposit', 15000, '2024-01-12'),
19 (107, 'B ANVESH', 'Vijayawada', 'Withdrawal', 1000, '2024-01-13'),
20 (108, 'G AASHRITHA', 'Hyderabad', 'Deposit', 9000, '2024-01-14'),
21 (109, 'D SAHASRA', 'Vizag', 'Withdrawal', 4000, '2024-01-15'),
22 (110, 'M SINDHU', 'Vijayawada', 'Deposit', 11000, '2024-01-16');
23
24 SELECT branch_name,
25        MAX(amount) AS Highest_Transaction
26 FROM bank_transactions
27 GROUP BY branch_name;
```

The right pane shows the output of the query. It includes a section for 'STDIN' (Input for the program) and an 'Output' section displaying the results of the SQL query in a table format:

branch_name	Highest_Transaction
Hyderabad	9000.00
Vijayawada	12000.00
Vizag	15000.00

Q14 — GROUP BY + MIN()

GROUP BY + MIN() — Find the Lowest Transaction in Each Branch



The screenshot shows the OneCompiler IDE with a MySQL query editor. The query defines a table named `bank_transactions` with columns: `txn_id` (INT PRIMARY KEY), `customer_name` (VARCHAR(50)), `branch_name` (VARCHAR(50)), `transaction_type` (VARCHAR(20)), `amount` (DECIMAL(10,2)), and `transaction_date` (DATE). It then inserts 10 rows of transaction data. Finally, it executes a query to find the lowest transaction amount for each branch using `GROUP BY` and `MIN()`.

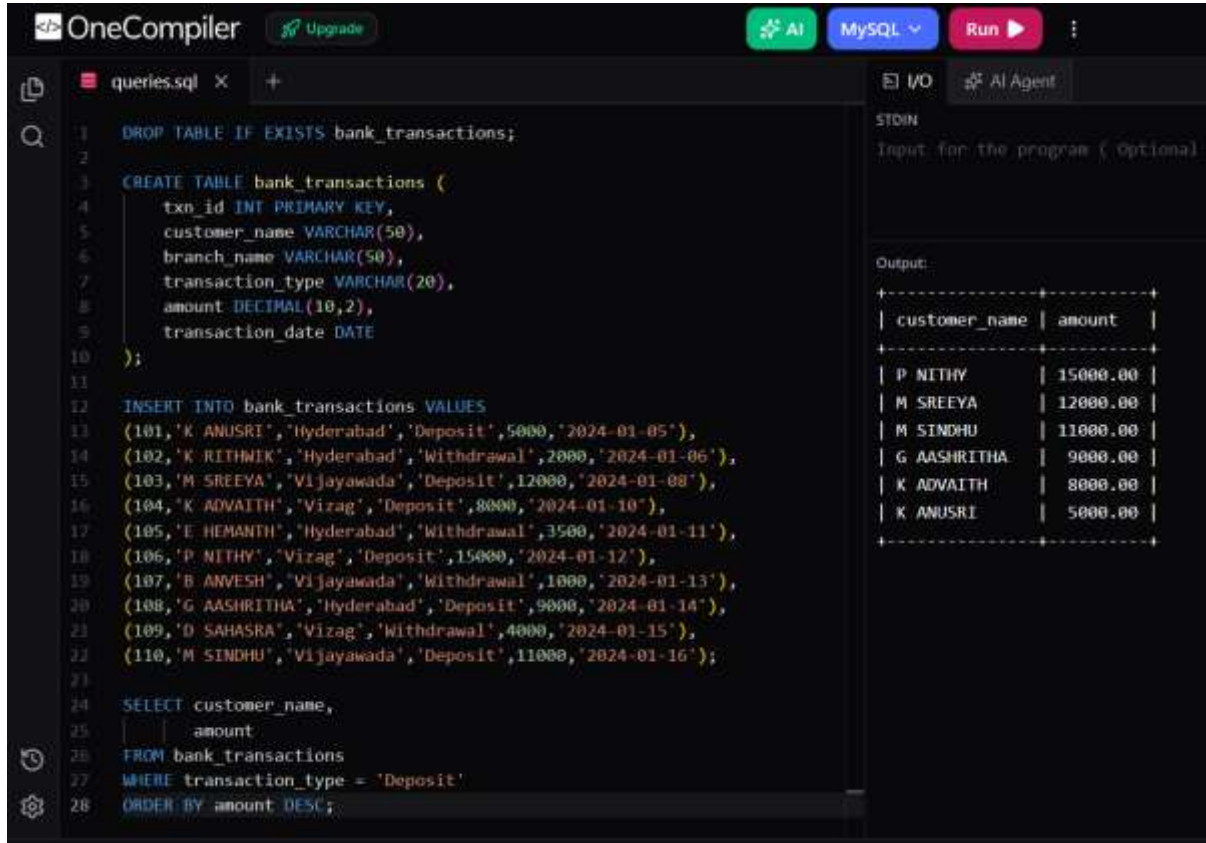
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101, 'K. ANUSRI', 'Hyderabad', 'Deposit', 5000, '2024-01-05'),
14 (102, 'K. RITHMIK', 'Hyderabad', 'Withdrawal', 2000, '2024-01-06'),
15 (103, 'M. SREEYA', 'Vijayawada', 'Deposit', 12000, '2024-01-08'),
16 (104, 'K. ADVAITH', 'Vizag', 'Deposit', 8000, '2024-01-10'),
17 (105, 'E. HEMANTH', 'Hyderabad', 'Withdrawal', 3500, '2024-01-11'),
18 (106, 'P. RITHY', 'Vizag', 'Deposit', 15000, '2024-01-12'),
19 (107, 'B. ANVESH', 'Vijayawada', 'Withdrawal', 1000, '2024-01-13'),
20 (108, 'G. AASHRITHA', 'Hyderabad', 'Deposit', 9000, '2024-01-14'),
21 (109, 'D. SAHASRA', 'Vizag', 'Withdrawal', 4000, '2024-01-15'),
22 (110, 'M. SINDHU', 'Vijayawada', 'Deposit', 11000, '2024-01-16');
23
24 SELECT branch_name,
25        MIN(amount) AS lowest_transaction
26 FROM bank_transactions
27 GROUP BY branch_name;
```

The output of the query is displayed in a table format:

branch_name	Lowest_Transaction
Hyderabad	2000.00
Vijayawada	1000.00
Vizag	4000.00

Q15 — WHERE + ORDER BY

WHERE + ORDER BY — Display Deposits from Highest to Lowest



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in 'queries.sql' with the following content:

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT customer_name,
25        amount
26 FROM bank_transactions
27 WHERE transaction_type = 'Deposit'
28 ORDER BY amount DESC;
```

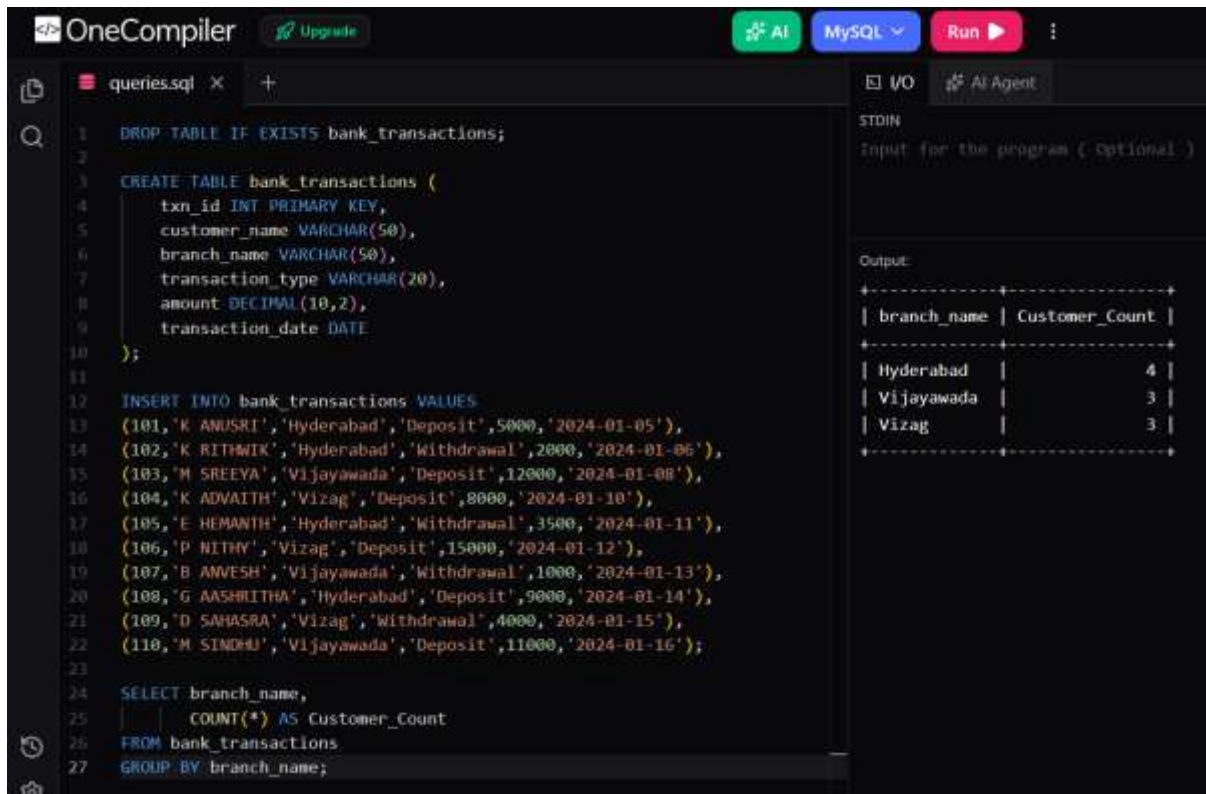
The right pane shows the output of the query, displaying a table with two columns: 'customer_name' and 'amount'. The data is sorted in descending order of amount.

customer_name	amount
P NITHY	15000.00
M SREEYA	12000.00
M SINDHU	11000.00
G AASHRITHA	9000.00
K ADVAITH	8000.00
K ANUSRI	5000.00

Q16 — GROUP BY + COUNT()

GROUP BY + COUNT() — Find Number of Customers

Transacting in Each Branch



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in a file named 'queries.sql'. The script creates a table 'bank_transactions' with columns: 'txn_id' (INT PRIMARY KEY), 'customer_name' (VARCHAR(50)), 'branch_name' (VARCHAR(50)), 'transaction_type' (VARCHAR(20)), 'amount' (DECIMAL(10,2)), and 'transaction_date' (DATE). It then inserts 10 rows of transaction data. Finally, it runs a query to select 'branch_name' and 'COUNT(*) AS Customer_Count' from 'bank_transactions', grouped by 'branch_name'.

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101, 'K ANUSRI', 'Hyderabad', 'Deposit', 5000, '2024-01-05'),
14 (102, 'K RITHVIK', 'Hyderabad', 'Withdrawal', 2000, '2024-01-06'),
15 (103, 'M SREEYA', 'Vijayawada', 'Deposit', 12000, '2024-01-08'),
16 (104, 'K ADVAITH', 'Vizag', 'Deposit', 8000, '2024-01-10'),
17 (105, 'E HEMANTH', 'Hyderabad', 'Withdrawal', 3500, '2024-01-11'),
18 (106, 'P NITHY', 'Vizag', 'Deposit', 15000, '2024-01-12'),
19 (107, 'B ANVESH', 'Vijayawada', 'Withdrawal', 1000, '2024-01-13'),
20 (108, 'G AASHLITHA', 'Hyderabad', 'Deposit', 9000, '2024-01-14'),
21 (109, 'D SAHASRA', 'Vizag', 'Withdrawal', 4000, '2024-01-15'),
22 (110, 'M SINDHU', 'Vijayawada', 'Deposit', 11000, '2024-01-16');
23
24 SELECT branch_name,
25        COUNT(*) AS Customer_Count
26 FROM bank_transactions
27 GROUP BY branch_name;
```

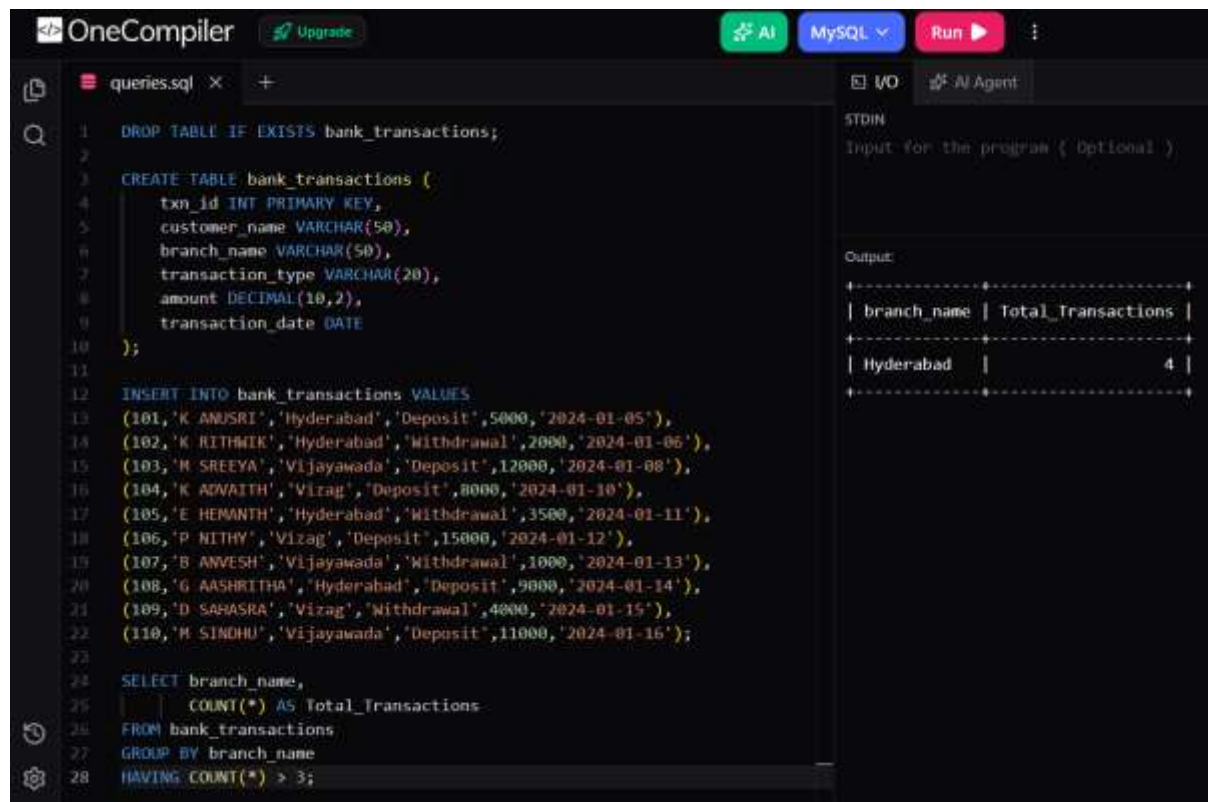
The right pane shows the output of the query. It displays a table with two columns: 'branch_name' and 'Customer_Count'. The data is as follows:

branch_name	Customer_Count
Hyderabad	4
Vijayawada	3
Vizag	3

Q17 — HAVING + COUNT()

HAVING + COUNT() — Find Branches That

Processed More Than 3 Transactions



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script in a file named 'queries.sql'. The script defines a 'bank_transactions' table, inserts 10 transactions, and queries for branches with more than 3 transactions. The right pane shows the output of the query, which is a table with two columns: 'branch_name' and 'Total_Transactions'. The output shows 'Hyderabad' with a total of 4 transactions.

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101, 'K. ANUSRI', 'Hyderabad', 'Deposit', 5000, '2024-01-05'),
14 (102, 'K. RITHMIK', 'Hyderabad', 'Withdrawal', 2000, '2024-01-06'),
15 (103, 'M. SREEYA', 'Vijayawada', 'Deposit', 12000, '2024-01-08'),
16 (104, 'K. ADVAITH', 'Vizag', 'Deposit', 8000, '2024-01-10'),
17 (105, 'E. HEMANTH', 'Hyderabad', 'Withdrawal', 3500, '2024-01-11'),
18 (106, 'P. NITHY', 'Vizag', 'Deposit', 15000, '2024-01-12'),
19 (107, 'B. ANVESH', 'Vijayawada', 'Withdrawal', 1000, '2024-01-13'),
20 (108, 'G. AASHRITHA', 'Hyderabad', 'Deposit', 9000, '2024-01-14'),
21 (109, 'D. SAHASRA', 'Vizag', 'Withdrawal', 4000, '2024-01-15'),
22 (110, 'M. SINDHU', 'Vijayawada', 'Deposit', 11000, '2024-01-16');
23
24 SELECT branch_name,
25        COUNT(*) AS Total_Transactions
26 FROM bank_transactions
27 GROUP BY branch_name
28 HAVING COUNT(*) > 3;
```

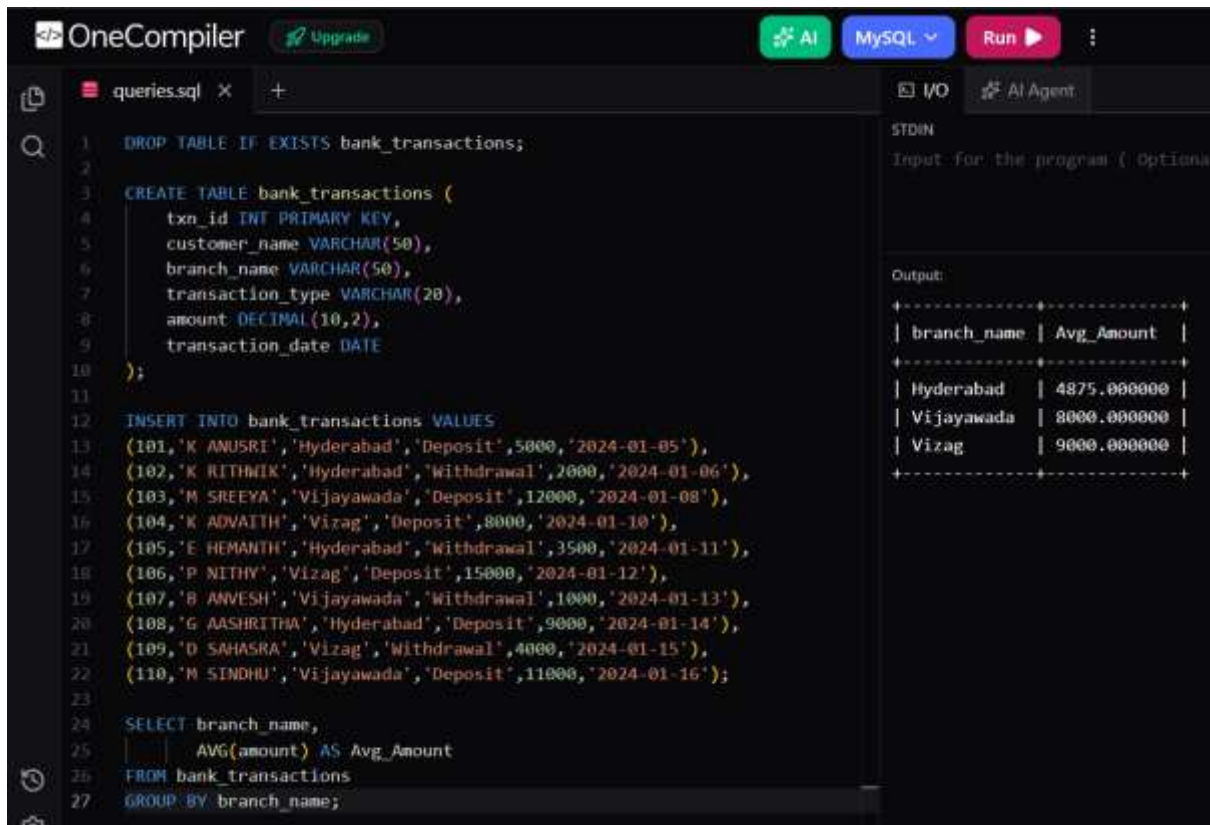
Output:

branch_name	Total_Transactions
Hyderabad	4

Q18 — GROUP BY + AVG()

GROUP BY + AVG() — Find Average Transaction

Amount for Each Branch



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script in 'queries.sql' with the following code:

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT branch_name,
25        AVG(amount) AS Avg_Amount
26 FROM bank_transactions
27 GROUP BY branch_name;
```

The right pane shows the output of the query:

STDIN
Input for the program (Optional):

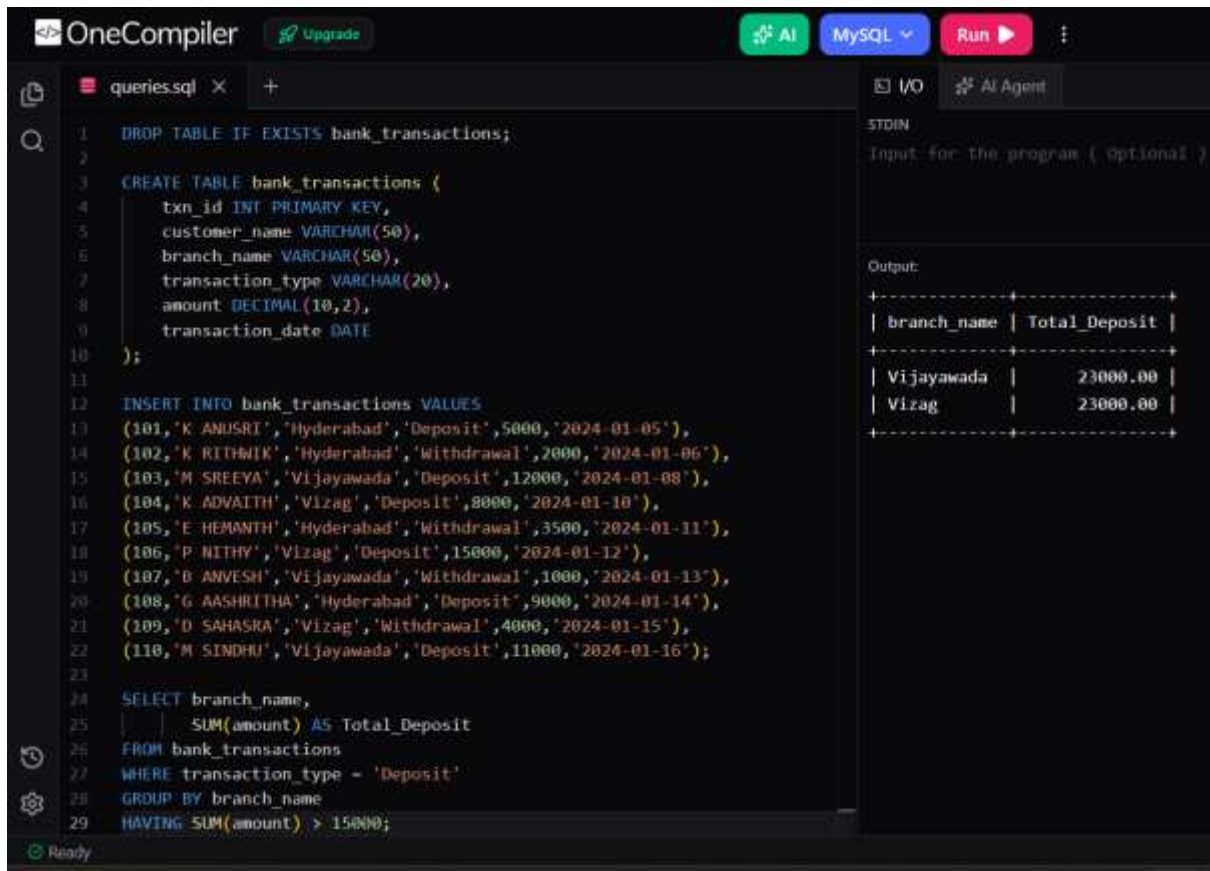
Output:

branch_name	Avg_Amount
Hyderabad	4875.000000
Vijayawada	8000.000000
Vizag	9000.000000

Q19 — WHERE + GROUP BY + HAVING

WHERE + GROUP BY + HAVING — Find Branches

Whose Total Deposit Amount Exceeds \$15,000



The screenshot shows the OneCompiler MySQL IDE. The main editor displays SQL code for creating a table, inserting data, and querying it. The right sidebar shows the output of the query, which is a table with two columns: branch_name and Total_Deposit. The output shows two rows: Vijayawada with a total deposit of 23000.00 and Vizag with a total deposit of 23000.00.

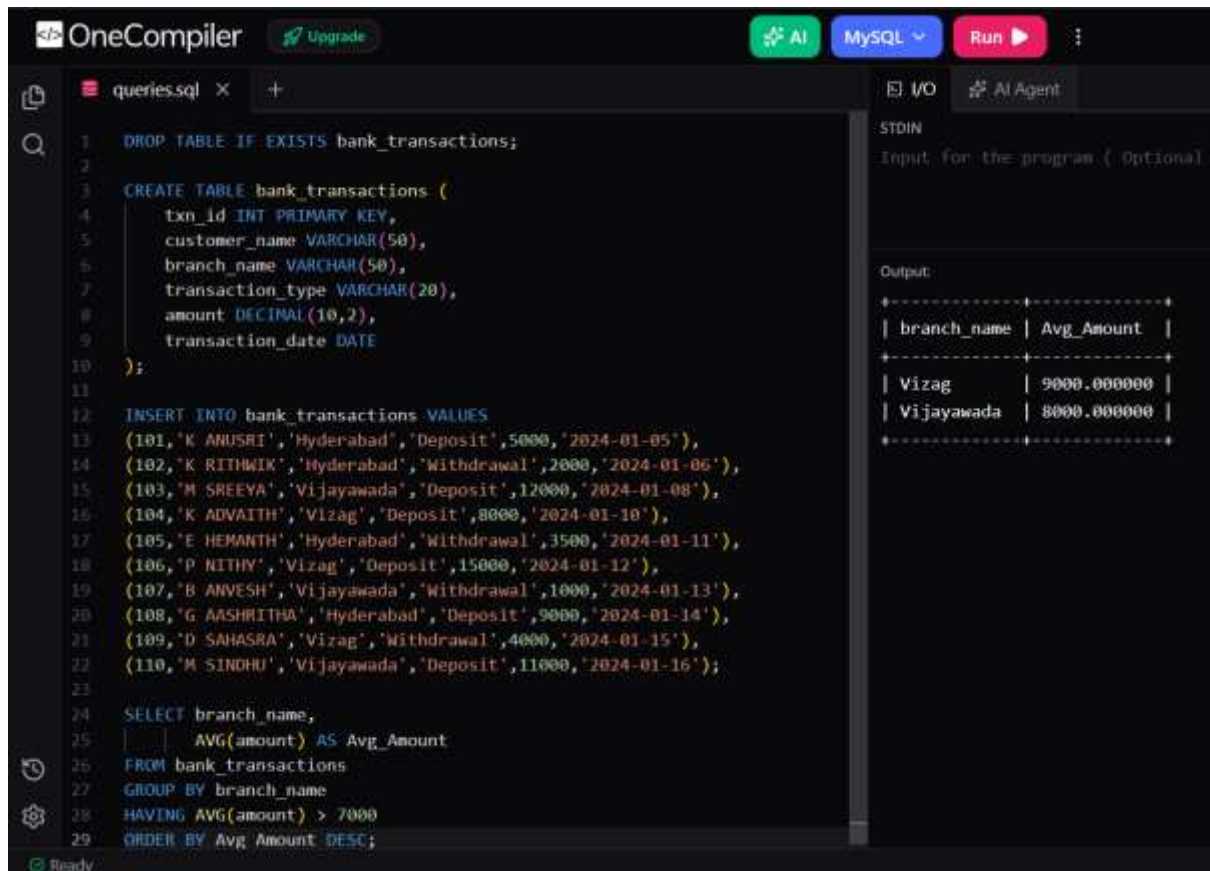
```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHVIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT branch_name,
25        SUM(amount) AS Total_Deposit
26 FROM bank_transactions
27 WHERE transaction_type = 'Deposit'
28 GROUP BY branch_name
29 HAVING SUM(amount) > 15000;
```

Output:

branch_name	Total_Deposit
Vijayawada	23000.00
Vizag	23000.00

Q20 — GROUP BY + HAVING + ORDER BY
GROUP BY + HAVING + ORDER BY — Find Branches

with Average Transaction Above \$7,000



The screenshot shows the OneCompiler MySQL IDE interface. The main editor displays a SQL script with the following queries:

```
1 DROP TABLE IF EXISTS bank_transactions;
2
3 CREATE TABLE bank_transactions (
4     txn_id INT PRIMARY KEY,
5     customer_name VARCHAR(50),
6     branch_name VARCHAR(50),
7     transaction_type VARCHAR(20),
8     amount DECIMAL(10,2),
9     transaction_date DATE
10 );
11
12 INSERT INTO bank_transactions VALUES
13 (101,'K ANUSRI','Hyderabad','Deposit',5000,'2024-01-05'),
14 (102,'K RITHWIK','Hyderabad','Withdrawal',2000,'2024-01-06'),
15 (103,'M SREEYA','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'K ADVAITH','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'E HEMANTH','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'P NITHY','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'B ANVESH','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'G AASHRITHA','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'D SAHASRA','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'M SINDHU','Vijayawada','Deposit',11000,'2024-01-16');
23
24 SELECT branch_name,
25        AVG(amount) AS Avg_Amount
26 FROM bank_transactions
27 GROUP BY branch_name
28 HAVING AVG(amount) > 7000
29 ORDER BY Avg_Amount DESC;
```

The right sidebar shows the output of the queries. The 'Output' section displays a table with the results of the final query:

branch_name	Avg_Amount
Vizag	9000.000000
Vijayawada	8000.000000

